PROBLEM STATEMENT

Create a Instrument Showroom Management System using Python, GUI, and MySQL to manage records of the showroom. perform,

- Connect to a MySQL database.
- Insert new records of shopping .
- Fetch and display records on a structured format.
- Add shopping records to a display
- Calculate the total bill dynamically.
- Clear all the records
- Update the records

INTRODUCTION

The **Musical Instrument Store Management System with GUi** is a Python project .

Created to help manage orders for musical instruments. It has a simple and easy-to-use interface made with tkinter, where customers can choose instruments, enter quantities, and apply discounts. The system automatically calculates the total bill and saves all order details in a MySQL database. You can view, update, or delete orders as needed. It also shows order details, including the bill, in the command line for quick reference. This project is perfect for small music stores or anyone looking to manage instrument sales efficiently.

OBJECTIVES

1. Establish a connection with the MySQL database to retrieve and manage instrument data.
2. Display a list of available Indian musical instruments with their prices in the Tkinter GUI.
3. Allow users to select an instrument and specify the quantity for purchase.
4. Validate user input to ensure correct quantity selection and display error messages if needed.
5. Show selected instruments with their quantities and total cost before checkout.
6. Implement a checkout process to confirm the purchase and clear the selection.
7. Ensure proper database connection handling and safe application exit.

FUNCTIONALITIES OF THE SYSTEM

Functionalities of the Musical Instrument Management System

1. Instrument Management

- Add, update, delete, and display musical instruments.
- Store instrument details such as name, price, and availability in the database.

2. User Selection & Purchase

- Allow users to browse and select instruments.
- Enable quantity selection for each instrument.

3. Cart Management

- Temporarily store selected instruments before purchase.
- Allow users to view, update, or remove items from the cart.

4. Checkout & Billing

- Calculate total cost based on selected items and quantity.
- Confirm purchases and generate an order summary.

5. Database Management

- Store and retrieve instrument details from a MySQL database.
- Ensure data consistency and integrity.

6. User-Friendly GUI

- Provide an interactive interface using Tkinter.
- Use buttons, labels, and list boxes for easy navigation.

7. Search & Filter

- Allow users to search for specific instruments.
- Implement filtering options based on price or category.

8. Error Handling & Validation

- Ensure valid quantity selection and prevent incorrect inputs.
- Display user-friendly error messages for invalid actions.

9. Secure Exit & Connection Handling

- Properly close database connections when exiting the application.
- Ensure smooth and error-free termination of the program.

User Interface

1. **Main Window:** Displays available instruments in a list with their names and prices, along with buttons for adding to the cart, viewing the cart, checkout, and exiting.

2. **Instrument Display Section:** Lists instruments retrieved from the database in a scrollable format for easy selection.

3. **Cart View Window:** Shows selected instruments with quantity, price, and total cost, along with options to remove items or proceed to checkout.

4. **Checkout Window:** Displays the final order summary, confirms the purchase, and clears the cart after successful checkout.

5. **Error Handling & Notifications:** Pop-up messages for invalid input, successful actions, and exit confirmation.

DATABASE NAME : ins_shop

TABLES : (ID , Customer , Phone , Email , Instruments ,Qty, Discount , Order_Date, Delivery_Date, Payment ,Status)

CREATING TABLE :

```
CREATE TABLE orders (
    ID INT PRIMARY KEY,
    Customer VARCHAR(255),
    Phone VARCHAR(15),
    Email VARCHAR(255),
    Instruments VARCHAR(255),
    Qty INT,
    Discount DECIMAL(5, 2),
    Order_Date DATE,
    Delivery_Date DATE,
    Payment VARCHAR(50),
    Status VARCHAR(50)
);
```

CODING

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector

# Connect to MySQL Database
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",  # Add your MySQL password if set
    database="musical_store"
)
cursor = conn.cursor()

# Create Table if not exists
cursor.execute('''
    CREATE TABLE IF NOT EXISTS orders (
        id INT AUTO_INCREMENT PRIMARY KEY,
        customer_name VARCHAR(255),
        phone_number VARCHAR(20),
        email VARCHAR(255),
        instruments VARCHAR(255),
        quantity INT,
        discount FLOAT,
        order_date DATE,
        delivery_date DATE,
        payment_method VARCHAR(50),
        payment_status VARCHAR(50)
    )
''')
conn.commit()
```

```
# GUI Application
root = tk.Tk()
root.title("Musical Instrument Store")
root.geometry("500x700")

font_style = ("Arial", 12)

# Labels & Entry Fields
tk.Label(root, text="Customer Name:", font=font_style).grid(row=0, column=0,
sticky="w")
customer_name_entry = tk.Entry(root)
customer_name_entry.grid(row=0, column=1)

tk.Label(root, text="Phone Number:", font=font_style).grid(row=1, column=0,
sticky="w")
phone_entry = tk.Entry(root)
phone_entry.grid(row=1, column=1)

tk.Label(root, text="Email:", font=font_style).grid(row=2, column=0, sticky="w")
email_entry = tk.Entry(root)
email_entry.grid(row=2, column=1)

# Instrument Selection
instrument_vars = {}
instruments = {"Veena": 50000, "Tabla": 30000, "Sitar": 71000, "Mridangam":
65000, "Flute": 15000}
```

```
tk.Label(root, text="Select Instrument:", font=font_style).grid(row=3, column=0,
sticky="w")
row_index = 4
for instr, price in instruments.items():
    var = tk.IntVar()
    instrument_vars[instr] = var
    tk.Checkbutton(root, text=f"{instr} - ₹{price}", variable=var,
font=font_style).grid(row=row_index, column=0, sticky="w")
    row_index += 1

# Quantity & Discount
tk.Label(root, text="Quantity:", font=font_style).grid(row=row_index, column=0,
sticky="w")
quantity_entry = tk.Entry(root)
quantity_entry.grid(row=row_index, column=1)
row_index += 1

tk.Label(root, text="Discount (%):", font=font_style).grid(row=row_index,
column=0, sticky="w")
discount_entry = tk.Entry(root)
discount_entry.grid(row=row_index, column=1)
row_index += 1

# Order & Delivery Date
tk.Label(root, text="Order Date (YYYY-MM-DD):",
font=font_style).grid(row=row_index, column=0, sticky="w")
order_date_entry = tk.Entry(root)
order_date_entry.grid(row=row_index, column=1)
row_index += 1
```

```
tk.Label(root, text="Delivery Date (YYYY-MM-DD):",
font=font_style).grid(row=row_index, column=0, sticky="w")
delivery_date_entry = tk.Entry(root)
delivery_date_entry.grid(row=row_index, column=1)
row_index += 1

# Payment Method
tk.Label(root, text="Payment Method:", font=font_style).grid(row=row_index,
column=0, sticky="w")
payment_method_var = ttk.Combobox(root, values=["Cash", "Card", "Online"])
payment_method_var.grid(row=row_index, column=1)
row_index += 1

# Payment Status
tk.Label(root, text="Payment Status:", font=font_style).grid(row=row_index,
column=0, sticky="w")
payment_status_var = ttk.Combobox(root, values=["Pending", "Paid"])
payment_status_var.grid(row=row_index, column=1)
row_index += 1

# Bill Section
tk.Label(root, text="Bill:", font=font_style).grid(row=row_index, column=0,
sticky="w")
bill_label = tk.Label(root, text="₹0.00", font=font_style)
bill_label.grid(row=row_index, column=1, sticky="w")
row_index += 1
```

```
# Database Operations
def add_order():
    selected_instruments = [instr for instr, var in instrument_vars.items() if var.get()
== 1]
    if not selected_instruments:
        messagebox.showwarning("Warning", "Please select at least one
instrument.")
        return

    cursor.execute("INSERT INTO orders (customer_name, phone_number, email,
instruments, quantity, discount, order_date, delivery_date, payment_method,
payment_status) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)",
                   (customer_name_entry.get(), phone_entry.get(), email_entry.get(), ",
".join(selected_instruments),
                    quantity_entry.get(), discount_entry.get(), order_date_entry.get(),
delivery_date_entry.get(),
                    payment_method_var.get(), payment_status_var.get()))
    conn.commit()
    messagebox.showinfo("Success", "Order placed successfully!")

def display_orders():
    cursor.execute("SELECT * FROM orders")
    orders = cursor.fetchall()
```

```
    # Print header
    print("\n{:<3} | {:<15} | {:<12} | {:<25} | {:<20} | {:<3} | {:<8} | {:<12} | {:<12} |
{:<10} | {:<8}".format(
        "ID", "Customer", "Phone", "Email", "Instruments", "Qty", "Discount",
"Order Date", "Delivery Date", "Payment", "Status"
    ))
    print("-" * 140)

    # Print each order
    for order in orders:
        order = list(order)
        order[7] = str(order[7])  # Convert order_date to string
        order[8] = str(order[8])  # Convert delivery_date to string
        print("{:<3} | {:<15} | {:<12} | {:<25} | {:<20} | {:<3} | {:<8} | {:<12} | {:<12} |
{:<10} | {:<8}".format(*order))

def update_order():
    cursor.execute("UPDATE orders SET payment_status='Paid' WHERE
phone_number=%s", (phone_entry.get(),))
    conn.commit()
    messagebox.showinfo("Success", "Payment Status Updated!")

def delete_order():
    cursor.execute("DELETE FROM orders WHERE phone_number=%s",
(phone_entry.get(),))
    conn.commit()
    messagebox.showinfo("Success", "Order Deleted!")
```

```
def clear_all_orders():
    confirm = messagebox.askyesno("Confirm", "Are you sure you want to delete
all orders?")
    if confirm:
        cursor.execute("DELETE FROM orders")
        conn.commit()
        messagebox.showinfo("Success", "All orders deleted successfully!")

# Calculate Bill
def calculate_bill():
    selected_instruments = [instr for instr, var in instrument_vars.items() if var.get()
== 1]
    quantity = int(quantity_entry.get()) if quantity_entry.get() else 1
    discount = float(discount_entry.get()) if discount_entry.get() else 0

    # Calculate total price of selected instruments
    total_price = sum(instruments[instr] for instr in selected_instruments) *
quantity

    # Apply discount
    total_price_after_discount = total_price - (total_price * (discount / 100))

    # Update the bill label
    bill_label.config(text=f"₹{total_price_after_discount:.2f}")
```

```
# Buttons
tk.Button(root, text="Place Order", command=add_order,
font=font_style).grid(row=row_index, column=0, pady=10, padx=5)
tk.Button(root, text="View Orders", command=display_orders,
font=font_style).grid(row=row_index, column=1, pady=10, padx=5)
row_index += 1

tk.Button(root, text="Update Order", command=update_order,
font=font_style).grid(row=row_index, column=0, pady=10, padx=5)
tk.Button(root, text="Delete Order", command=delete_order,
font=font_style).grid(row=row_index, column=1, pady=10, padx=5)
row_index += 1

tk.Button(root, text="Clear All Orders", command=clear_all_orders,
font=font_style, bg="red", fg="white").grid(row=row_index, column=0,
columnspan=2, pady=10, sticky="ew")
row_index += 1

tk.Button(root, text="Calculate Bill", command=calculate_bill,
font=font_style).grid(row=row_index, column=0, columnspan=2, pady=10,
sticky="ew")

# Close MySQL connection on exit
def on_closing():
    conn.close()
    root.destroy()

root.protocol("WM_DELETE_WINDOW", on_closing)

root.mainloop()
```

OUTPUT

Musical Instrument Store

Customer Name: Preethi

Phone Number: 56885634324

Email: preethi@gmail.com

Select Instrument:

- [x] Veena - ₹50000
- [ ] Tabla - ₹30000
- [ ] Sitar - ₹71000
- [ ] Mridangam - ₹65000
- [ ] Flute - ₹15000

Quantity: 1

Discount (%): 27

Order Date (YYYY-MM-DD): 2020-5-12

Delivery Date (YYYY-MM-DD): 2020-5-21

Payment Method: Card

Payment Status: Paid

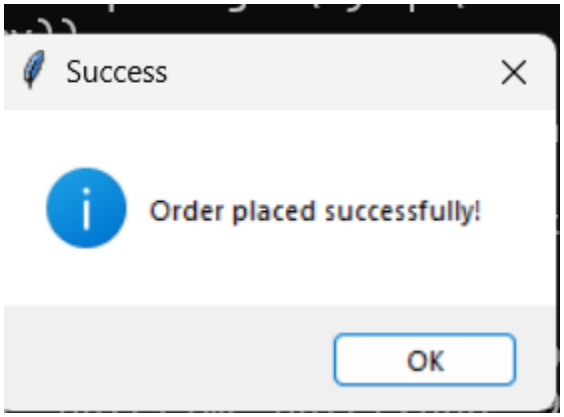
Bill: ₹36500.00

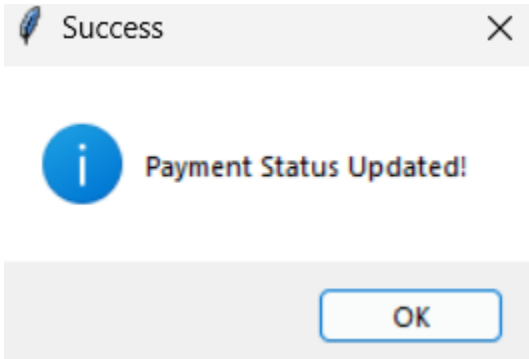
Place Order | View Orders

Update Order | Delete Order

Clear All Orders

Calculate Bill





ID	Customer	Phone	Email	Instruments	Qty	Discount	Order Date	Delivery Date	Payment	Status

Update:

ID	Customer	Phone	Email	Instruments	Qty	Discount	Order Date	Delivery Date	Payment	Status
12	tanishka	9443876594	tanishka@gmail.com	Tabla	2	29.0	2005-06-09	2005-06-19	Card	Paid
13	dwarshika	23678956	dwarshika@gmail.com	Veena	1	20.0	2021-03-09	2021-03-10	Cash	Paid
14	sudha	23678956	sudha@gmail.com	Flute	1	20.0	2021-03-08	2021-03-18	Cash	Paid
15	sudha	23678956	sudha@gmail.com	Flute	1	25.0	2021-03-08	2021-03-18	Online	Paid

Inserting extra records:

```
ID  | Customer        | Phone        | Email                    | Instruments          | Qty | Discount | Order Date   | Delivery Date | Payment    | Statu
s
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
12  | tanishka        | 9443876594   | tanishka@gmail.com       | Tabla                | 2   | 29.0     | 2005-06-09   | 2005-06-19    | Card       | Paid

13  | dwarshika       | 23678956     | dwarshika@gmail.com      | Veena                | 1   | 20.0     | 2021-03-09   | 2021-03-10    | Cash       | Paid

14  | sudha           | 23678956     | sudha@gmail.com          | Flute                | 1   | 20.0     | 2021-03-08   | 2021-03-18    | Cash       | Paid

15  | sudha           | 23678956     | sudha@gmail.com          | Flute                | 1   | 25.0     | 2021-03-08   | 2021-03-18    | Online     | Paid

16  | sachitha        | 23673487     | sachitha@gmail.com       | Sitar                | 1   | 30.0     | 2022-07-01   | 2022-07-19    | Online     | Paid

17  | rabeela         | 23673598     | rabeela@gmail.com        | Sitar                | 1   | 30.0     | 2022-07-09   | 2022-07-19    | Card       | Paid
```

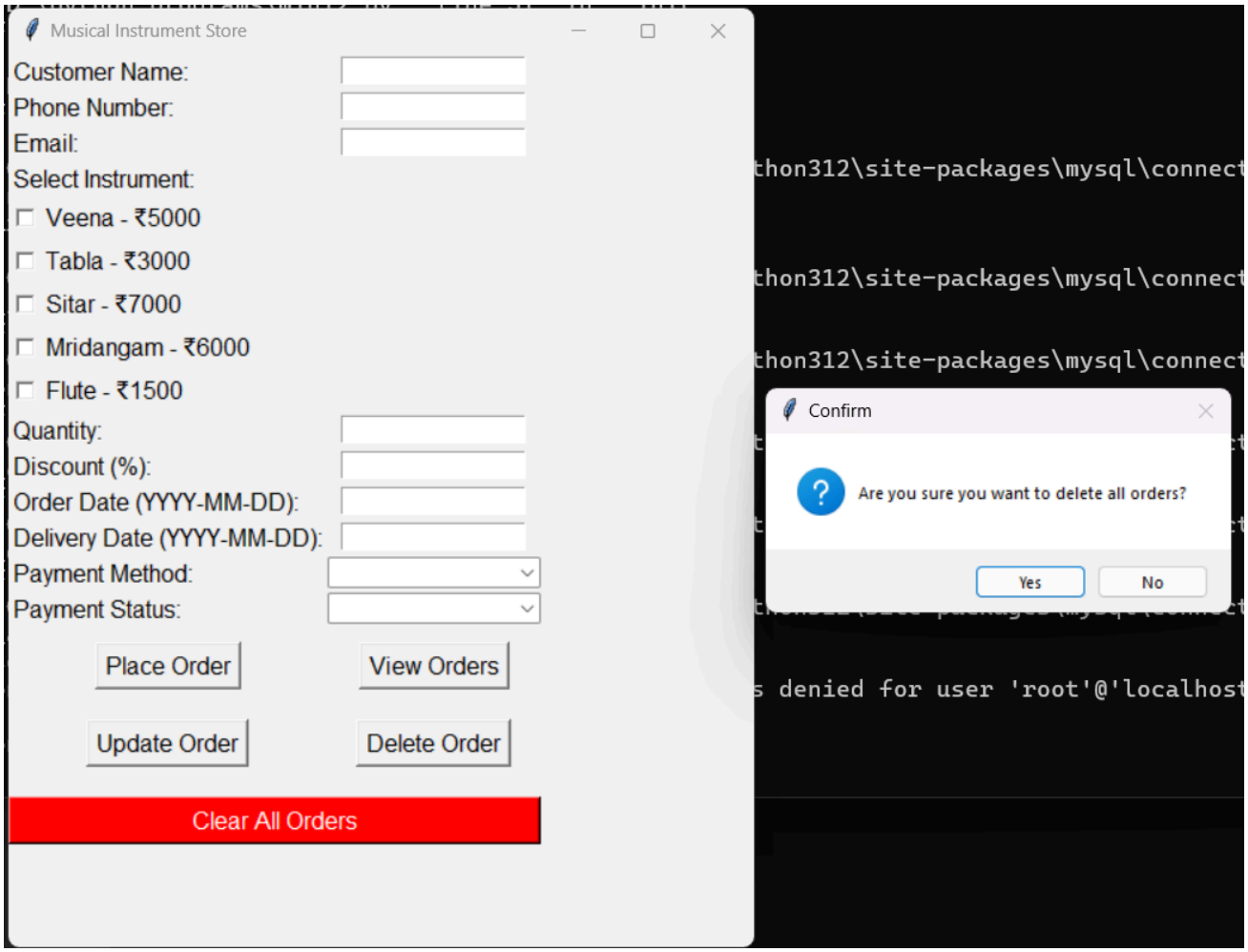
```
D:\python programs>python sample.py

ID  | Customer        | Phone        | Email                    | Instruments          | Qty | Discount | Order Date   | Delivery Date | Payment    | Statu
s
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
22  | Preethi         | 56885634324  | preethi@gmail.com        | Veena                | 1   | 27.0     | 2020-05-12   | 2020-05-21    | Card       | Paid
```

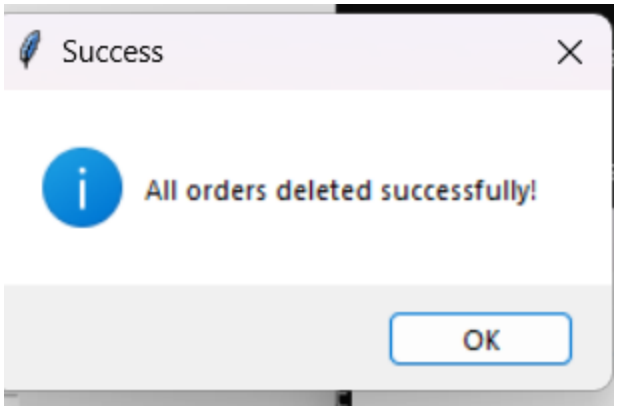
View orders:

```
ID  | Customer        | Phone        | Email                    | Instruments          | Qty | Discount | Order Date   | Delivery Date | Payment    | Statu
s
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
12  | tanishka        | 9443876594   | tanishka@gmail.com       | Tabla                | 2   | 29.0     | 2005-06-09   | 2005-06-19    | Card       | Paid

13  | dwarshika       | 23678956     | dwarshika@gmail.com      | Veena                | 1   | 20.0     | 2021-03-09   | 2021-03-10    | Cash       | Paid

14  | sudha           | 23678956     | sudha@gmail.com          | Flute                | 1   | 20.0     | 2021-03-08   | 2021-03-18    | Cash       | Paid

15  | sudha           | 23678956     | sudha@gmail.com          | Flute                | 1   | 25.0     | 2021-03-08   | 2021-03-18    | Online     | Paid

16  | sachitha        | 23673487     | sachitha@gmail.com       | Sitar                | 1   | 30.0     | 2022-07-01   | 2022-07-19    | Online     | Paid

17  | rabeela         | 23673598     | rabeela@gmail.com        | Sitar                | 1   | 30.0     | 2022-07-09   | 2022-07-19    | Card       | Paid
```

Deleting all records:

```
ID  | Customer        | Phone        | Email                    | Instruments          | Qty | Discount | Order Date   | Delivery Date | Payment    | Statu
s
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
```


Musical Instrument Store
Customer Name:
Phone Number:
Email:
Select Instrument:
Veena - ₹5000
Tabla - ₹3000
Sitar - ₹7000
Mridangam - ₹6000
Flute - ₹1500
Quantity:
Discount (%):
Order Date (YYYY-MM-DD):
Delivery Date (YYYY-MM-DD):
Payment Method:
Payment Status:
Place Order
View Orders
Update Order
Delete Order
Clear All Orders
thon312\site-packages\mysql\connect
thon312\site-packages\mysql\connect
thon312\site-packages\mysql\connect
s denied for user 'root'@'localhost
Confirm
Are you sure you want to delete all orders?
Yes
No




```
D:\python programs>python mini.py

ID  | Customer   | Phone        | Email                    | Instruments     | Qty | Discount | Order Date   | Delivery
 Date | Payment    | Status
-------------------------------------------------------------------------------------------------------------------------------
-------------------
```

Conclusion

The Instrument Showroom project successfully demonstrates the integration of a MySQL database with a user-friendly GUI to manage inventory, sales, and customer data efficiently. By enabling seamless operations such as adding, updating, deleting, and retrieving instrument details, the project ensures data integrity and optimized performance. The intuitive GUI enhances user interaction, while features like inventory tracking and sales management highlight its practicality. Designed with scalability in mind, the project addresses challenges such as data consistency and GUI responsiveness, showcasing a strong understanding of database management and frontend development. This project serves as a robust solution for instrument showroom operations, with potential for future enhancements like online sales or advanced analytics. Overall, it reflects a successful implementation of technical skills and provides a solid foundation for further improvements.